CS 289 / ROB 201 · WEEK 1

How a robot knows where things are

Introduction to Robot Perception & Spatial Geometry, with Lab 1: 2D & 3D Rigid Transformations

Dr.-Ing. Chukwuka Victor Obionwu · 2 hours

THE PLAN

Two hours, four quizzes, one lab

Time	Block	Format
0:00 – 0:12	Why perception, the pipeline, coordinate frames	Lecture
0:12 – 0:30	Planar rotations, SO(2)	Lecture + Quiz 1
0:30 – 0:55	SE(3): compose, invert, order matters	Lecture + Quiz 2
0:55 – 1:00	Break	Stretch
1:00 – 1:25	Lab 1: starter code, warm-up, pose chaining	Live coding
1:25 – 1:40	Numerical drift and the Boss Level exp map	Live coding
1:40 – 1:55	Lab quizzes 3 and 4	Quiz
1:55 – 2:00	Recap, deadline, next week	Wrap-up

Four skills, and why each one matters

1 · Name the pipeline

Why: a failing robot is a failing stage. Knowing the four stages tells you where to look first.

2 · Move points between frames

Why: camera, LiDAR, IMU and arm each place their origin somewhere else.

3 · Build SE(3) poses

Why: one 4×4 matrix carries rotation and translation, so chaining poses is a multiplication.

4 · Keep rotations valid

Why: tiny floating-point errors pile up and quietly corrupt a long pose chain.

HOOK

A camera sees a cup at pixel (412, 230). The arm needs it in metres.

Camera frame
The cup is 0.6 m in front of the lens.

→

Robot base frame
The camera is bolted on, tilted and offset.

→

Gripper frame
The arm only knows its own joints.

Without frame transformations, perception is a pretty picture. With them, it is an action.

THE ROBOT PERCEPTION PIPELINE

From photons to a map in four stages

STAGE 1

Sensing and preprocessing

Camera frames, LiDAR clouds, IMU data. Fix lens distortion, filter noise.

STAGE 2

Feature extraction and association

Keypoints, edges, descriptors, matched across consecutive time steps.

STAGE 3

Spatial geometry reconstruction

Triangulate 3D points and relative poses with epipolar constraints.

STAGE 4

State estimation and optimization

EKF and factor graphs fuse vision with inertial data into a consistent map.

Today lives in stages 3 and 4. Both are written in the language of frames, rotations and poses.

COORDINATE FRAMES

Every sensor, actuator and landmark has its own

THE RIGID TRANSFORMATION

$$p_A = R_{A \leftarrow B}\, p_B + t_{A \leftarrow B}$$

p_B
The point as measured in frame B.

R in SO(3)
How frame B is rotated relative to A.

t in R^3
Where the origin of B sits, written in A.

WORKED EXAMPLE · PLANAR

A landmark seen by the robot, placed in the world

Given

Robot frame B is rotated 90° from world A, and its origin sits at (1, 2) in A.

The robot sees a landmark at (2, 1) in its own frame.

Solve pA = R pB + t

$$R(90^\circ) \cdot (2, 1) = (-1, 2)$$

$$(-1, 2) + (1, 2) = (0, 4)$$

The landmark is at **(0, 4)** in the world frame. Rotate first, then translate.

A planar robot turns about the vertical axis

ROTATION MATRIX

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Counter-clockwise by θ about the origin.

LAB CODE

```
def rotation_matrix_2d(theta):
  c, s = np.cos(theta), np.sin(theta)
  return np.array([
    [c, -s],
    [s, c]
  ], dtype=np.float64)
```

Check it: θ = 45° gives 0.7071 on the diagonal, ±0.7071 off it. R(90°) sends (1, 0) to (0, 1).

WHAT MAKES A ROTATION VALID

Orthogonal, determinant +1, inverse is the

$R^TR = I$

Columns are unit length and perpendicular. Lengths and angles survive.

$\det R = +1$

Rules out mirror flips, which have determinant −1.

$R^{-1} = R^T$

No matrix inversion needed. Transpose is free.

Composition: $R(30°) \cdot R(60°) = R(90°)$. In the plane, angles simply add.

Verified numerically. This tidy behaviour is special to 2D.

Pick the best answer

Q1. Which pipeline stage fuses vision with inertial data using an EKF or factor graph?

A. Sensing and preprocessing

B. Feature extraction

C. Geometry reconstruction

D. State estimation

Q2. In $p_A = R\,p_B + t$, what does p_B mean?

A. The point measured in frame B

B. The point measured in frame A

C. The translation vector

D. The rotation angle

Q3. What is R(90°) applied to the point (2, 1)?

A. (1, −2)

B. (−1, 2)

C. (2, −1)

D. (−2, −1)

QUIZ 1 · ANSWERS 1 TO 3

D, A, B

Q1 · D

State estimation and optimization is stage 4: EKF and factor graphs fuse visual odometry with inertial telemetry into a consistent map.

Q2 · A

The superscript names the frame in which the point is measured. p^B is in B, and the transform produces p^A.

Q3 · B

$x' = 0 \cdot 2 - 1 \cdot 1 = -1$ and $y' = 1 \cdot 2 + 0 \cdot 1 = 2$, so $(-1, 2)$. Option A is the $-90°$ result.

Pick the best answer

Q4. R is a 90° rotation, t = (1, 2) and p_B = (2, 1). What is p_A?

A. (3, 3)

B. (−1, 2)

C. (0, 4)

D. (2, 1)

Q5. For any rotation matrix R, the inverse R^{-1} equals:

A. −R

B. R^T

C. R^2

D. det(R) · R

Q6. True or false: R(30°) · R(60°) = R(90°).

A. True

B. False

QUIZ 1 · ANSWERS 4 TO 6

C, B, True

Q4 · C

Rotate first: R(90°)(2, 1) = (−1, 2). Then add t: (−1, 2) + (1, 2) = (0, 4). Option B forgot the translation.

Q5 · B

Rotations are orthogonal, so $R^T R = I$ and the inverse is just the transpose. It is also $R(-\theta)$.

Q6 · True

In the plane, rotating 30° then 60° is the same as rotating 90°. Angles add, and the order does not matter in 2D.

WHY SE(3)

Rotate then translate is clumsy. Can we make it one multiplication?

Before

```
pA = R pB + t
```

Two operations. Chaining three frames means nested expressions that are easy to get wrong.

After

```
pA = T pB
```

One matrix. Chaining becomes T_AC = T_AB · T_BC.

The trick: add a fourth coordinate equal to 1. That is homogeneous coordinates.

HOMOGENEOUS COORDINATES

A pose is a 4×4 matrix in SE(3)

THE POSE MATRIX

```
T = [ R t ]
[ 0 0 0 1 ]
```

R is 3×3, t is 3×1, bottom row is 0 0 0 1.

THE POINT

```
p_h = [x, y, z, 1]□
```

Then pA = T pB does rotation and translation together.

In the lab: make_se3_transform starts from np.eye(4), writes R into T[:3, :3] and t into T[:3, 3]. The identity gives you the bottom row for free.

POSE COMPOUNDING

Chain frames by multiplying matrices

$$T_{A \leftarrow C} = T_{A \leftarrow B} \cdot T_{B \leftarrow C}$$

Given

T_AB: no rotation, shift (2, 0, 0).

T_BC: 90° about z, shift (0, 1, 0).

Point in C: (1, 0, 0).

Result

T_AC has translation (2, 1, 0).

The point lands at **(2, 2, 0)** in frame A.

Check: rotate (1, 0, 0) to (0, 1, 0), add (0, 1, 0), add (2, 0, 0).

THE CLASSIC BUG

A · B is not B · A

A = turn 90° about z
B = move 1 m along x

A · B ends at **(0, 1, 0)**

B · A ends at **(1, 0, 0)**

Same two moves, different robot position. Compounding transformations in the wrong order is one of the most common bugs in robotics software.

GOING BACKWARDS

Inverting a pose without a general matrix inverse

$$T^{-1} = \begin{bmatrix} R^{\top} & -R^{\top} t \\ 0\ 0\ 0 & 1 \end{bmatrix}$$

Example

T: rotate 90° about z, translate (1, 2, 3).

T^{-1} has translation **(−2, 1, −3)**.

Check

$T^{-1} \cdot T$ equals the identity.

T^{-1} sends (1, 2, 4) back to **(0, 0, 1)**.

The translation is not simply −t. You must undo the rotation as well.

LECTURE RECAP

Four takeaways before the second quiz

Perception bridges

Raw sensor signals become spatial world models.

Frames need book-keeping

Always say which frame a point lives in.

SE(3) chains poses

4×4 matrices make 3D rigid motion a product.

Verify your rotations

Check $R^T R = I$ and $\det R = 1$, always.

QUIZ 2 · QUESTIONS 1 TO 3 · SE(3)

Pick the best answer

Q1. What size is an SE(3) homogeneous transformation matrix?

A. 3×3

B. 3×4

C. 4×4

D. 6×6

Q2. The homogeneous form of the 3D point (x, y, z) is:

A. [x, y, z, 0]

B. [x, y, z, 1]

C. [1, x, y, z]

D. [x, y, z]

Q3. Which expression gives $T_{A \leftarrow C}$ from $T_{A \leftarrow B}$ and $T_{B \leftarrow C}$?

A. $T_{AB} + T_{BC}$

B. $T_{BC} \cdot T_{AB}$

C. $T_{AB} \cdot T_{BC}$

D. $T_{AB} \cdot T_{BC}^{-1}$

QUIZ 2 · ANSWERS 1 TO 3

C, B, C

Q1 · C

A 3×3 rotation plus a 3×1 translation plus a bottom row 0 0 0 1 makes a 4×4 matrix.

Q2 · B

The extra coordinate is 1, which lets the fourth column of T add the translation.

Q3 · C

T_AC = T_AB · T_BC. The inner B cancels. Option B has the order reversed.

QUIZ 2 · QUESTIONS 4 TO 6 · SE(3)

Pick the best answer

Q4. True or false: $T_{AB} \cdot T_{BC} = T_{BC} \cdot T_{AB}$ for any two poses.

A. True

B. False

Q5. T = 90° about z, t = (1, 2, 3). Where does T send the point (1, 0, 0)?

A. (2, 2, 3)

B. (1, 3, 3)

C. (0, 2, 3)

D. (1, 2, 4)

Q6. For the same T, what is the translation part of T^{-1}?

A. (−1, −2, −3)

B. (−2, 1, −3)

C. (2, −1, 3)

D. (1, −2, 3)

QUIZ 2 · ANSWERS 4 TO 6

False, B, B

Q4 · False

Matrix multiplication is non-commutative. A turn plus a shift ends in a different place depending on order.

Q5 · B

R(90°)(1, 0, 0) = (0, 1, 0). Add t = (1, 2, 3) to get (1, 3, 3).

Q6 · B

$-R^T t$: R^T rotates (1, 2, 3) to (2, −1, 3), so the translation is (−2, 1, −3). Option A forgot to rotate.

BREAK · 5 MINUTES

Stretch, then open your editor

When we return: write the three Lab 1 functions together. Have Python 3 and NumPy ready.

Think about this while you stand: if a robot turns then walks, versus walks then turns, which one ends further from home?

Three levels, from a matrix to a trajectory

WARM-UP

2D rotation

Implement rotation_matrix_2d and test it on known vector rotations.

CORE

SE(3) pose chaining

Chain poses for a simulated robot along a 5-waypoint 3D trajectory.

BOSS LEVEL

Exponential map

Implement exp(ϕ^) from so(3) to SO(3) and compare compute time with Euler conversion.

Due Sunday, 11:59 PM EST. Commit and push to your GitHub Classroom repository.

STARTER CODE · PART 1

Build the rotation and the pose matrix

rotation_matrix_2d

```
c, s = np.cos(theta), np.sin(theta)
return np.array([
  [c, -s],
  [s, c]
], dtype=np.float64)
```

make_se3_transform

```
T = np.eye(4, dtype=np.float64)
T[:3, :3] = R
T[:3, 3] = np.asarray(t).flatten()
return T
```

Why np.eye(4)? It pre-fills the bottom row 0 0 0 1.

Why flatten()? t may arrive as 3×1 or as three numbers.

STARTER CODE · PART 2

Transform many points at once

```
ones = np.ones((num_points, 1), dtype=points.dtype)
points_h = np.hstack([points, ones])
transformed_h = (T @ points_h.T).T
return transformed_h[:, :3]
```

points	points_h	points_h.T	T @ points_h.T	result
(N, 3)	(N, 4)	(4, N)	(4, N)	(N, 3)

Try it: a 90° turn about z with t = (1, 2, 3) sends (1, 0, 0) to (1, 3, 3) and (0, 1, 0) to (0, 2, 3).

WARM-UP

Test the rotation against answers you already know

Input point	Angle	Expected output
(1, 0)	90°	(0, 1)
(1, 0)	45°	(0.7071, 0.7071)
(2, 1)	90°	(−1, 2)
(3, 4)	90°	(−4, 3)
any point	180°	the point negated

Add two property checks: `np.allclose(R.T @ R, np.eye(2))` and a determinant of 1.

CORE TASK · POSE CHAINING

A robot walks a square, one pose multiplication at a

SKETCH

```
step = make_se3_transform(
  Rz(np.pi / 2), [1, 0, 0])
T = np.eye(4)
poses = [T]
for _ in range(4):
  T = T @ step
  poses.append(T)
```

EXPECTED POSITIONS

```
0: (0, 0, 0)
1: (1, 0, 0)
2: (1, 1, 0)
3: (0, 1, 0)
4: (0, 0, 0)
```

Pitfall: T @ step moves in the robot's own frame. step @ T applies the move in the world frame, and traces a different path.

NUMERICAL PRECISION NOTICE

Long chains drift. Repair rotations with SVD.

A corrupted R

$\|R^\top R - I\| \approx 0.0037$

$\det R \approx 0.9990$

Small noise of order 0.001 added to a valid rotation.

After SVD

$\|R^\top R - I\| \approx 1e{-}15$

$\det R = 1.0000$

The nearest valid rotation to the noisy matrix.

```
U, S, Vt = np.linalg.svd(R)
R_clean = U @ Vt
```

QR decomposition works too. If det comes out −1, flip the sign of one column first.

BOSS LEVEL · PART 1

One vector in so(3) becomes a rotation in SO(3)

RODRIGUES FORMULA

$$\theta = \|\phi\|, \quad K = \text{hat}(\phi / \theta)$$

$$\exp(\phi^\wedge) = I + \sin\theta \cdot K + (1 - \cos\theta) \cdot K^2$$

READ IT

The direction of ϕ is the rotation axis. Its length is the angle in radians.

$\phi = (0, 0, \pi/2)$ gives exactly a 90° turn about z.

The result is always orthogonal with determinant 1, and it matches scipy.linalg.expm.

BOSS LEVEL · PART 2

Why bother? Compare with Euler angles

Gimbal lock

At pitch ±90°, roll and yaw act about the same axis, so different angle pairs give the same rotation. Checked numerically.

Benchmark it yourself

Time 20,000 conversions with time.perf_counter. Predict first, then measure, then explain.

Honest note: in my pure-Python run, Rodrigues took about twice as long as a simple Euler build. The exponential map earns its place through smoothness and no singularities, not necessarily raw speed.